

M.NAGA SHALINI
192524183

```
1.n = int(input("Enter number of hosts: "))
switch = "Switch"
print("\nStar Topology")
print("Central Device:", switch)
for i in range(1, n + 1):
    length = int(input("Enter cable length for Host " + str(i) + " (in meters): "))
    print("Host", i, "->", switch, "-> Cat6 UTP ->", length, "meters")
print("\nAll hosts are connected through the central switch.")
print("Star topology provides reliable communication because")
print("failure of one cable affects only that host.")
```

OUTPUT

```
Enter number of hosts: 3
Enter cable length for Host 1 (in meters): 10
Enter cable length for Host 2 (in meters): 20
Enter cable length for Host 3 (in meters): 15
Star Topology
Central Device: Switch
Host 1 -> Switch -> Cat6 UTP -> 10 meters
Host 2 -> Switch -> Cat6 UTP -> 20 meters
Host 3 -> Switch -> Cat6 UTP -> 15 meters
All hosts are connected through the central switch.
Star topology provides reliable communication because
failure of one cable affects only that host.
```

```
2.def validate_segment(length_m):
    if length_m <= 100 and length_m >= 0:
        return True, "Valid cable length"
    else:
        return False, "Error: Cable length must not exceed 100 meters"
lengths = [50, 100, 120]
for length in lengths:
    result, message = validate_segment(length)
    print(length, "meters:", result, "-", message)
```

OUTPUT

```
50 meters: True - Valid cable length
100 meters: True - Valid cable length
120 meters: False - Error: Cable length must not exceed 100 meters
```

```
3.import random
hosts = ["Host1", "Host2", "Host3", "Host4"]
log = open("csma_log.txt", "w")
for host in hosts:
    attempt = 1
    while True:
        log.write(host + " - Transmission attempt " + str(attempt) + "\n")
        collision = random.choice([True, False])
        if collision:
            backoff = random.randint(1, 5)
            print(host, "- Collision detected")
            print("Back-off time:", backoff)
            log.write("Collision detected\n")
            log.write("Back-off time: " + str(backoff) + "\n")
            attempt = attempt + 1
        else:
            print(host, "- Transmission successful")
            log.write("Transmission successful\n")
            break
```

```
log.close()
print("\nCSMA/CD simulation completed.")
print("Details are stored in csma_log.txt")
```

OUTPUT

```
Host1 - Collision detected
Back-off time: 3
Host1 - Transmission successful
Host2 - Transmission successful
Host3 - Collision detected
```

```

Back-off time: 2
Host3 - Transmission successful
Host4 - Transmission successful
CSMA/CD simulation completed.
Details are stored in csma_log.txt

```

```

4.import socket
server = socket.socket()
server.bind(("localhost", 5000))
server.listen(1)
print("Server waiting for connection...")
conn, addr = server.accept()
mac_table = {}
while True:
    data = conn.recv(1024).decode()
    if not data:
        break
    parts = data.split()
    host = parts[0]
    mac = parts[1]
    port = parts[2]
    mac_table[mac] = port
    print("\nFrame received from:", host)
    print("MAC Address:", mac)
    print("\nMAC Address Table")
    for m, p in mac_table.items():
        print(m, "->", p)
    conn.send("Frame received successfully".encode())
conn.close()
server.close()

```

CLIENT CODE

```

import socket
client = socket.socket()
client.connect(("localhost", 5000))
host = input("Enter host name: ")
mac = input("Enter MAC address: ")
port = input("Enter port number: ")
message = host + " " + mac + " " + port
client.send(message.encode())
response = client.recv(1024).decode()
print(response)
client.close()

```

OUTPUT

```

Server waiting for connection...
Frame received from: Host1
MAC Address: AA:BB:CC:DD:EE:01
MAC Address Table
AA:BB:CC:DD:EE:01 -> 1

```

```

5.class StarSwitch:
    def __init__(self):
        self.ports = []
        self.log = []
    def add_port(self, host_id):
        self.ports.append(host_id)
        print(host_id, "connected")
    def remove_port(self, host_id):
        if host_id in self.ports:
            self.ports.remove(host_id)
            print(host_id, "disconnected")
    def broadcast(self, frame, source):
        print("\nBroadcasting frame:", frame)
        print("Source:", source)
        receivers = []
        for host in self.ports:
            if host != source:
                receivers.append(host)
                print("Frame received by:", host)
        self.log.append((source, receivers))

```

```
switch = StarSwitch()
switch.add_port("Host1")
switch.add_port("Host2")
switch.add_port("Host3")
switch.add_port("Host4")
switch.broadcast("Hello", "Host1")
switch.remove_port("Host4")
print("\nActive Ports:")
print(switch.ports)
print("\nBroadcast Log:")
for entry in switch.log:
    print("Source:", entry[0], "Receivers:", entry[1])
```

OUTPUT

```
Host1 connected
Host2 connected
Host3 connected
Host4 connected
Broadcasting frame: Hello
Source: Host1
Frame received by: Host2
Frame received by: Host3
Frame received by: Host4
Host4 disconnected
Active Ports:
['Host1', 'Host2', 'Host3']
Broadcast Log:
Source: Host1 Receivers: ['Host2', 'Host3', 'Host4']
```